SPRING DATA JPA IMPLEMENTATION

(Entity → Repository → Service → Controller)

Spring Data JPA is one of the most **frequently used** and **most misunderstood** parts of Spring Boot. Many developers can write repositories, but very few can explain **how save(), findAll(), pagination, and query methods actually work internally**.

Interviewers use this topic to separate people who "used Spring Boot" from those who **understand how Spring Boot works**. If you can clearly explain repository hierarchy, internal implementations, and query resolution, you immediately stand out as a strong backend engineer.

Project Setup – JOB Application Management System

We start by creating a Spring Boot project using Spring Initializr. The selected dependencies are Spring Web for REST APIs, Spring Data JPA for persistence, Lombok to reduce boilerplate code, and H2 Database for an in-memory database. H2 is intentionally chosen because it allows us to focus on JPA behavior without external database setup. This setup mirrors how interview demos and PoCs are typically built.

Spring Boot auto-configures DataSource, EntityManagerFactory, TransactionManager, and Repository infrastructure without us writing configuration classes. This is important to remember, because interviewers often ask what Spring Boot configures automatically when JPA is added.

Entity Layer – Applicant Entity

We start with a simple POJO and then convert it into a JPA Entity. A JPA Entity represents a database table, and each object of that class represents a row. When Spring Boot starts, Hibernate scans entity classes, builds metadata, and prepares SQL mappings internally.

```
package com.example.jobportal.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "applicant")
```

```java
@Entity
@Table(name = "applicant")
public class Applicant {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String name;
    private String email;
    private String phone;
    private String status;

    public Applicant() {}

    public Long getId() {
        return id;
    }
```

```java
  public String getName() {
      return name;
  }

  public void setName(String name) {
      this.name = name;
  }

  public void setId(Long id) {
      this.id = id;
  }

  public String getEmail() {
      return email;
  }
```

```
    public void setEmail(String email) {
        this.email = email;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

@Id and @GeneratedValue – Generation Strategy Deep Dive

The @Id annotation tells JPA that this field is the primary key.
The @GeneratedValue annotation tells JPA **how the primary key should be generated**.

AUTO lets Hibernate choose the best strategy based on the database.
IDENTITY uses database auto-increment columns.
SEQUENCE uses database sequences and is preferred in enterprise databases like Oracle and PostgreSQL.
TABLE uses a separate table to maintain IDs and is rarely used due to performance issues.
UUID generates globally unique identifiers and is useful in distributed systems.

Repository Layer – CrudRepository

The first repository we create extends **CrudRepository**.

```
package com.example.jobportal.repository;

import com.example.jobportal.entity.Applicant;
import org.springframework.data.repository.CrudRepository;

public interface ApplicantCrudRepository extends CrudRepository<Applicant, Long> {
}
```

CrudRepository is an interface, but the implementation is **not written by us**.

Spring Data generates a proxy class at runtime. This proxy delegates calls to an internal implementation backed by EntityManager.

When you call save(), Spring Data converts it into a JPA persist or merge operation.

CrudRepository provides basic operations like save, findById, findAll, deleteById. It returns Iterable, which is why additional conversion logic is often needed in services.

Service Layer – Business Logic with CrudRepository

The service layer sits between controller and repository. It contains business logic and orchestration. This separation is important for testing and future scalability.

```
package com.example.jobportal.service;

import com.example.jobportal.entity.Applicant;
import com.example.jobportal.repository.ApplicantCrudRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
```

```java
@Service
public class ApplicantService {

    @Autowired
    private ApplicantCrudRepository applicantCrudRepository;

    public Applicant saveApplicantCrud(Applicant applicant) {
        return applicantCrudRepository.save(applicant);
    }

    public List<Applicant> getAllApplicants() {
        Iterable<Applicant> iterable = applicantCrudRepository.findAll();
        List<Applicant> applicants = new ArrayList<>();
        iterable.forEach(applicants::add);
        return applicants;
    }
}
```

Here, save() internally checks whether the entity has an ID. If the ID is null, Hibernate performs an insert. If the ID exists, Hibernate performs an update. This decision happens inside the persistence context, not inside your code.

Controller Layer – API Entry Point

The controller exposes REST endpoints and delegates business logic to the service layer.

```java
package com.example.jobportal.controller;

import com.example.jobportal.entity.Applicant;
import com.example.jobportal.service.ApplicantService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
```

```java
@RestController
@RequestMapping("/applicants")
public class ApplicantController {

    @Autowired
    private ApplicantService applicantService;

    @GetMapping
    public List<Applicant> getAllApplicants() {
        return applicantService.getAllApplicants();
    }

    @PostMapping
    public Applicant saveApplicant(@RequestBody Applicant applicant) {
        return applicantService.saveApplicantCrud(applicant);
    }
}
```

Request Flow – What Happens Internally

When a POST request is sent from Postman, it reaches the embedded Tomcat server. Tomcat forwards the request to DispatcherServlet. DispatcherServlet finds the correct controller method using HandlerMapping. The JSON body is converted into an Applicant object using Jackson. The controller calls the service, which calls the repository. The repository proxy delegates to Hibernate, which executes SQL against H2. The response flows back the same path.

This flow explanation is extremely important in interviews.

PagingAndSortingRepository – Pagination Support

PagingAndSortingRepository extends CrudRepository and adds pagination and sorting support.

```java
public interface ApplicantPagingAndSortingRepository
        extends PagingAndSortingRepository<Applicant, Long> {
}
```

This repository allows fetching data in chunks instead of loading everything into memory. Internally, Spring Data converts pagination requests into SQL with LIMIT and OFFSET.

```
public List<Applicant> getApplicantsWithPagination(int page, int size) {
    return applicantPagingAndSortingRepository
            .findAll(PageRequest.of(page, size))
            .getContent();
}
```

Pagination is critical for performance and scalability. Interviewers expect you to justify why pagination exists.

ListCrudRepository and ListPagingAndSortingRepository

ListCrudRepository is a newer interface that returns List instead of Iterable. This removes boilerplate conversion logic in services.

```
public interface ApplicantListCrudRepository
        extends ListCrudRepository<Applicant, Long> {
}
```

Similarly, ListPagingAndSortingRepository combines pagination with List returns. These interfaces improve developer experience but internally still rely on the same Spring Data infrastructure.

Repository Hierarchy – Who Does What

CrudRepository provides basic CRUD.
PagingAndSortingRepository adds pagination and sorting.
JpaRepository extends PagingAndSortingRepository and adds JPA-specific features.

Actual implementations are provided by SimpleJpaRepository, which uses EntityManager internally. This is a very common interview question.

JpaRepository – Advanced JPA Operations

```
public interface ApplicantJpaRepository
        extends JpaRepository<Applicant, Long> {
}
```

JpaRepository adds methods like flush, saveAndFlush, deleteInBatch, findAll with sorting, and getById. These methods interact directly with the persistence context and transaction boundaries. flush() forces SQL execution immediately instead of waiting for transaction commit.

Query Methods – Why They Exist

Spring Data allows writing queries by method name. This removes boilerplate JPQL for common queries.

```
List<Applicant> findByStatus(String status);
```

Spring parses the method name, builds a JPQL query internally, and executes it. This is done at startup time, not runtime, which means errors are detected early.

Custom Query Methods with @Query

Some queries cannot be expressed using method names. In those cases, @Query is used.

```
@Query("SELECT a FROM Applicant a WHERE a.name LIKE %:name%")
List<Applicant> findApplicantsByPartialName(@Param("name") String name);
```

Here, JPQL is used instead of SQL. JPQL works with entity names and fields, not table names. Spring binds parameters safely and prevents SQL injection.

Service and controller methods simply delegate to these repository methods, keeping business logic clean.

Interview Takeaway

If you can explain **repository hierarchy, internal implementations, save behavior, pagination, and query resolution**, you are already ahead of most candidates. Spring Data JPA is not magic — it is **well-structured abstraction over EntityManager**, and interviews are all about understanding that abstraction.
